Monomorphization Optimizer — Report

Kamyak Channa

April 2026

1 Optimization Description

What was implemented

The optimizer is a Soot-based whole-program transformer that *devirtualizes* Java virtual and interface method calls when an interprocedural type-flow analysis can prove that the receiver at a given call site is always of exactly one concrete class. A proven-monomorphic call is rewritten from `invokevirtual`/`invokeinterface` to `invokestatic`, dispatched to a freshly synthesized static clone of the concrete target method.

The baseline transformer already handled calls where the receiver flowed through locals, parameters, casts, `new`, and invoke-results. This work extends both the analysis and the rewriter along three dimensions that were previously left conservative:

1. **Instance and static field references** — types written through a `FieldRef` are recorded into a global per-field map, and reads from fields populate the local's type set from that map.
2. **Array references** — writes to `ArrayRef` contribute to a global map keyed by the *element type*, and reads produce that set.
3. **Interface invokes** — both `VirtualInvokeExpr` and `InterfaceInvokeExpr` are now analyzed and rewritten.

Why it is useful

Virtual dispatch in Java is expensive for three reasons: a hidden vtable/itable lookup on every call, a cache-unfriendly memory indirection, and the fact that the JIT cannot inline an unresolved virtual callee without a guard. Static analysis that collapses a call to a single concrete target turns the call into a plain `invokestatic` — faster to dispatch even in the pure interpreter, and structurally inlineable once the JIT sees it. In practice, many Java programs use `List<T>`, strategy objects held in fields, or arrays of a single concrete subclass; without field/array tracking, the analysis is blind to those shapes, so the original optimizer missed large classes of common patterns (e.g. fluent `StringBuilder.append` chains devirtualize only once field types are known).

2 Analyses Used

Type-flow analysis

The core abstraction is a forward dataflow analysis over Soot's Jimple CFG:

- **Lattice element (`TypeState`):** `Map<Local, Set<SootClass>>`. An absent key means "no type information" (conservative $\bot$); an empty set means the local is known to be untyped. Join is point-wise set union.
- **Transfer function** (`TypeFlowAnalysis.flowThrough`): `IdentityStmt` seeds parameters from the calling context; `NewExpr` produces a singleton; `Local := Local` propagates; `CastExpr` narrows via `FastHierarchy.canStoreClass`; `InvokeExpr` consumes the callee's

return-type summary for the current arg context; `FieldRef` and `ArrayRef` read from the global maps; writes to those locations update the globals.

Dimensions of precision

- **Flow-sensitive.** Type sets are maintained per program point, computed by Soot's `ForwardFlowAnalysis`.
- **Context-sensitive on method parameters** (1-CFA-like). The analyzer is parameterised by a `paramCtx: Map<Integer, Set<SootClass>>` keyed by parameter index (-1 for the receiver). The worklist enqueues `(method, ctx)` pairs; method summaries are *ctx* $\rightarrow$ *returnTypes*. Virtual targets are enqueued with the context refined to the specific receiver type, which is how per-dispatch-target precision is recovered.
- **Field-based** (not field-sensitive). A field is treated as a single abstract cell shared across all instances — `SootField` $\rightarrow$ `Set<SootClass>`. This trades some precision for simplicity and scales well to large heaps.
- **Array-element-type-based.** `Type` $\rightarrow$ `Set<SootClass>`, keyed by the declared element type (`ArrayRef.getType()`). All arrays with the same element type are merged.
- **Conservative on unmodelled rhs.** Any assignment whose right-hand side the analysis does not understand (constants, intrinsics it doesn't model) *clears* the lhs set, so stale types from a previous store can never survive.

Assumptions

- SPARK call graph, built on-the-fly (`-p cg.spark on-fly-cg:true`).
- Reflection is resolved via the TamiFlex `refl.log` that accompanies each DaCapo input directory.
- Only application classes (as seen by Soot, which — with `-app` — includes TamiFlex-dumped JDK classes) are analyzed.
- **JDK rewriting is disabled.** `$mono` clones are only created when the concrete target's declaring class is outside `java.*`, `javax.*`, `sun.*`, `jdk.*`, `com.sun.*`. See §3 for why.
- Correctness hinges on **outer fixed-point iteration**: the field/array global maps are re-checked after each worklist pass, and a new pass is started if either map grew. Monotonicity (maps only grow) guarantees termination in a small number of passes.

3 Transformation Details

Before / after snippets

Field-flowing call (interface dispatch through an instance field). Before monomorphization (excerpt of `Kennel.call`):

```
$r1 = r0.<Kennel: Animal resident>;
$r2 = interfaceinvoke $r1.<Animal: String speak()>();
return $r2;
```

After:

```
$r1 = r0.<Kennel: Animal resident>;
$mono_cast_0 = (Dog) $r1;
$r2 = staticinvoke <Dog: String speak$mono$Dog(Dog)>($mono_cast_0);
return $r2;
```

The `interfaceinvoke` is replaced by an `invokestatic` to a synthesized `speak$mono$Dog`, a static copy of `Dog.speak` whose first parameter is the receiver. An explicit checkcast is inserted because `$r1`'s declared type (`Animal`) is wider than the static clone's first parameter type (`Dog`).

Array-element-flowing call. Before:

```
r5 = r0[i3];                // r0: Shape[]
$i1 = interfaceinvoke r5.<Shape: int area()>();
```

After:

```
r5 = r0[i3];
$mono_cast_0 = (Square) r5;
$i1 = staticinvoke <Square: int area$mono$Square(Square)>(
    $mono_cast_0);
```

Generated static clone (added as a new method on the concrete class):

```
static String speak$mono$Dog(Dog) {
    Dog r0;
    r0 := @parameter0: Dog;    // originally: @this: Dog
    return "woof";
}
```

Identity statements are fixed up when cloning: `@this: T` becomes `@parameter0: T`, and each existing `@parameterN: U` is shifted to `@parameter(N+1): U`.

Key challenges

1. **Cross-method field/array data flow.** Writers and readers of a field can be in different methods; the worklist visits methods in arbitrary order, so a reader may be analyzed before any writer has been seen. We solved this with a monotone outer fixed-point: the `(method, ctx)` worklist is re-seeded until the global field and array maps stop growing. In practice two or three passes suffice even on DaCapo-scale input, because the maps are small.

2. **Field-sensitive vs. field-based.** True instance-field sensitivity requires pointer-analysis-grade heap abstractions. We deliberately chose the field-based abstraction (one abstract cell per field symbol). This loses precision when a field can legitimately hold different types in different instances, but it keeps the analysis fast and is sound — any over-approximation leaves the call site conservatively virtual.

3. **Interface invokes were silently ignored by the original rewriter.** Fixing field-s/arrays without fixing interface invokes would have hidden the benefit, because typical field-loaded receivers are declared with an interface type (`Animal`, `Op`, `Shape`). Both the analyser's receiver-type collection and the rewriter's instance-invoke dispatch were generalized to handle `InterfaceInvokeExpr`.

4. **Receiver type mismatch after devirtualization.** A call site typed `Animal a; a.speak()` passes a local declared as `Animal`, but the clone's first parameter is declared as the concrete class (`Dog`). The JVM verifier rejects this mismatch. The rewriter now inserts a `CastExpr` assign into a fresh local immediately before the call when the base local's declared type is not equal to the concrete type.

5. **JDK class rewriting breaks at runtime.** With `-app` Soot treats TamiFlex-dumped JDK classes as application classes, and the original rewriter happily synthesized `$mono` clones on `java.lang.StringBuilder` and friends. Emitted classes live under `sootOutput/`, but at runtime the JVM loads `java.*` from the boot class-loader, not from our output directory — so every rewritten call site would fail with `NoSuchMethodError`. The rewriter

now skips any concrete target whose declaring class is under a JDK package prefix. The *analysis* still traverses JDK methods, so JDK return types still refine application call sites; only the rewrite is suppressed.

4 Result Tables

4.1 Functional correctness

Four purpose-built micro testcases exercise the new capabilities. For each, the original bytecode (JVM baseline) and the monomorphized bytecode (after applying the transformer, then running the emitted `.class` files) are executed and their stdout compared.

Test case	Exercised feature	JVM	After mono	Match	Sites
`field_case`	interface dispatch via instance field	woof	woof	✓	3
`array_case`	interface dispatch via array element	29	29	✓	1
`mixed_case`	array held inside an instance field	RR	RR	✓	5
`static_field_case`	static field written/read across methods	hello	hello	✓	1

Table 1: Functional correctness: all outputs match the JVM baseline.

Disassembly (`javap -c`) confirms each devirtualized call site is an `invokestatic` to a `$mono` clone and not an `invokeinterface`/`invokevirtual`.

4.2 Performance

Microbenchmarks (`-Xint`, 4×10^8 iterations, median of 5)

Unit: seconds wall-clock. Workload = tight loop dispatching through the feature under test.

Microbench	Sites	Baseline (s)	Optimized (s)	Δ	Speedup
`bench_array`	1	7.86	6.89	+12.3%	1.14×
`bench_mixed`	2	8.50	8.56	−0.7%	0.99×
`bench_field`	3	12.28	13.07	−6.4%	0.94×

Table 2: Micro-benchmarks, pure interpreter mode.

Under `-Xint` the only saving is `invokeinterface`→`invokestatic` (one fewer hash-table lookup per call); the inserted `checkcast` costs about as much in the pure interpreter. `bench_array` is a clean win because the dispatch sits in the hottest possible loop and the cast is cheap compared to an interface lookup; `bench_field` shows a small regression because three rewritten sites include `Holder.run` whose original `virtualinvoke` was already fast. The transformation is structured to feed the JIT, which inlines the static clone and folds the cast away — that benefit is invisible under `-Xint`.

DaCapo 9.12 benchmarks (`-s default`, `-Xint`, 3 runs each)

Unit: seconds wall-clock. "Sites" is the number of application-level call sites the transformer rewrote (JDK targets deliberately skipped).

(Per-run times, logs, and transformed class files are archived under `bench-results/`.)

Discussion

- `xalan` sees the largest genuine win (+11%) despite having the fewest rewritten sites: those 282 sites sit on the tight XSLT template-dispatch loop that dominates its interpreted

Benchmark	Sites	Baseline mean	Baseline min	Optimized mean	Optimized min	Speedup
`xalan`	282	4.65 s	4.55 s	4.19 s	4.15 s	**1.11**×
`avrora`	1 090	8.70 s	8.32 s	8.40 s	8.32 s	1.04×
`fop`	2 231	3.22 s	3.19 s	3.18 s	3.13 s	1.01×
`luindex`	1 279	5.16 s	5.05 s	5.19 s	5.06 s	0.99×

Table 3: DaCapo 9.12 benchmarks, JIT disabled. `batik` excluded because DaCapo-9.12 still requires `com.sun.image.codec.jpeg`, which was removed in JDK 9.

runtime.

- `luindex`'s interpreted time is spent mostly in JDK code paths (file channels, regular expressions, `StringBuilder`) — exactly the surface our JDK-skip filter refuses to rewrite, so there is nothing to gain.
- `avrora` and `fop` show small positive deltas (+1–4%); their hot paths are mixed app/JDK, so devirtualization dilutes proportionally.
- No benchmark regresses beyond $\sim 1\%$ noise, indicating the checkcast-insertion cost and the extra static-frame overhead are well-contained.

4.3 Reproduction

All numbers can be reproduced with the supplied driver script:

```
# default: luindex avrora xalan fop, 3 runs, -s default, JIT off
./bench.sh

# quick smoke test
SIZE=small RUNS=1 ./bench.sh

# subset with more runs
BENCHES="xalan avrora" RUNS=5 ./bench.sh
```

The script recompiles the transformer, applies it once per benchmark to emit a `mono-<bench>/` directory of `.class` files, then runs `java -Xint -cp dacapo.jar Harness -s default <bench>` `RUNS` times for the baseline and again with `mono-<bench>/` prepended to the classpath, emitting per-run times, means, mins, and speedup ratios to stdout and `bench-results/`.